

C Programming Assignment

Menu-Driven Digit-Based Operations Using switch-case

Problem Statement

Write a menu-driven C program using switch-case to perform digit-based operations on a positive integer. Define one separate user-defined function for each menu option, with void return type and a simple function call (without call by value/reference). Each function should read the required input, process it, and display the result.

- Check whether its digits are in increasing order from left to right.
- Check whether its digits are in decreasing order from left to right.
- Find the largest digit and the smallest digit.
- Find the second largest digit and second smallest digit in the number.

Algorithm

1. Start the program.
2. Display a menu containing four digit-based operations and an exit option.
3. Read the user's menu choice.
4. Use switch-case to call the corresponding void function.
5. Inside the selected function, read a positive integer from the user.
6. For increasing order, compare adjacent digits from right to left. The number is increasing from left to right only when every left digit is smaller than the digit to its right.
7. For decreasing order, compare adjacent digits from right to left. The number is decreasing from left to right only when every left digit is greater than the digit to its right.
8. For largest and smallest digits, repeatedly extract digits using modulo 10 and update the current largest and smallest values.
9. For second largest and second smallest digits, track the two largest distinct digits and two smallest distinct digits while processing the number.
10. Display the result of the selected operation.
11. Repeat the menu until the user selects Exit.
12. Stop the program.

C Program

```
#include <stdio.h>
```

```

void increasingOrder(void);
void decreasingOrder(void);
void largestSmallest(void);
void secondLargestSmallest(void);

int main(void)
{
    int choice;

    do
    {
        printf("\n--- DIGIT OPERATIONS MENU ---\n");
        printf("1. Check increasing order\n");
        printf("2. Check decreasing order\n");
        printf("3. Find largest and smallest digit\n");
        printf("4. Find second largest and second smallest digit\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: increasingOrder(); break;
            case 2: decreasingOrder(); break;
            case 3: largestSmallest(); break;
            case 4: secondLargestSmallest(); break;
            case 5: printf("Program terminated.\n"); break;
            default: printf("Invalid choice. Try again.\n");
        }
    } while (choice != 5);

    return 0;
}

void increasingOrder(void)
{
    long long n, temp;
    int right, left, increasing = 1;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);
    if (n <= 0) { printf("Please enter a positive integer.\n"); return; }

    temp = n;
    right = temp % 10;
    temp /= 10;
    while (temp > 0)
    {
        left = temp % 10;
        if (left >= right) { increasing = 0; break; }
        right = left;
        temp /= 10;
    }

    if (increasing)
        printf("Digits of %lld are in increasing order.\n", n);
    else
        printf("Digits of %lld are not in increasing order.\n", n);
}

void decreasingOrder(void)
{
    long long n, temp;
    int right, left, decreasing = 1;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

```

```

    if (n <= 0) { printf("Please enter a positive integer.\n"); return; }

    temp = n;
    right = temp % 10;
    temp /= 10;
    while (temp > 0)
    {
        left = temp % 10;
        if (left <= right) { decreasing = 0; break; }
        right = left;
        temp /= 10;
    }

    if (decreasing)
        printf("Digits of %lld are in decreasing order.\n", n);
    else
        printf("Digits of %lld are not in decreasing order.\n", n);
}

void largestSmallest(void)
{
    long long n, temp;
    int digit, largest = 0, smallest = 9;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);
    if (n <= 0) { printf("Please enter a positive integer.\n"); return; }

    temp = n;
    while (temp > 0)
    {
        digit = temp % 10;
        if (digit > largest) largest = digit;
        if (digit < smallest) smallest = digit;
        temp /= 10;
    }
    printf("Largest digit = %d\n", largest);
    printf("Smallest digit = %d\n", smallest);
}

void secondLargestSmallest(void)
{
    long long n, temp;
    int digit, largest = -1, secondLargest = -1;
    int smallest = 10, secondSmallest = 10;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);
    if (n <= 0) { printf("Please enter a positive integer.\n"); return; }

    temp = n;
    while (temp > 0)
    {
        digit = temp % 10;

        if (digit > largest)
        {
            secondLargest = largest;
            largest = digit;
        }
        else if (digit > secondLargest && digit != largest)
            secondLargest = digit;

        if (digit < smallest)
        {
            secondSmallest = smallest;
            smallest = digit;
        }
    }
}

```

```

        else if (digit < secondSmallest && digit != smallest)
            secondSmallest = digit;

        temp /= 10;
    }

    if (secondLargest == -1 || secondSmallest == 10)
        printf("The number does not contain at least two distinct digits.\n");
    else
    {
        printf("Second largest digit = %d\n", secondLargest);
        printf("Second smallest digit = %d\n", secondSmallest);
    }
}

```

Test Case Table

S.No.	Menu Choice	Input	Expected Output	Actual Output	Result
1	1	123456	Digits are in increasing order.	Digits of 123456 are in increasing order.	Pass
2	1	124356	Digits are not in increasing order.	Digits of 124356 are not in increasing order.	Pass
3	2	975310	Digits are in decreasing order.	Digits of 975310 are in decreasing order.	Pass
4	2	975510	Digits are not in decreasing order.	Digits of 975510 are not in decreasing order.	Pass
5	3	508274	Largest = 8, Smallest = 0	Largest digit = 8; Smallest digit = 0	Pass
6	4	583172	Second largest = 7, Second smallest = 2	Second largest digit = 7; Second smallest digit = 2	Pass
7	4	7777	At least two distinct digits are required.	The number does not contain at least two distinct digits.	Pass

Result

The menu-driven C program was compiled and tested for the required digit-based operations. The supplied outputs verify increasing order, decreasing order, largest/smallest digit, and second largest/second smallest digit operations. The repeated-digit test (7777) also verifies that the program correctly reports when at least two distinct digits are not present.

Execution Screenshots

Increasing-order test: 123456 (Pass)

```
— DIGIT OPERATIONS MENU —
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
5. Exit
Enter your choice: 1
Enter a positive integer: 123456
Digits of 123456 are in increasing order.
```

Increasing-order negative test: 124356 (Pass)

```
(kali@kali)-[~/hdoc16]
$ ./day16

— DIGIT OPERATIONS MENU —
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
5. Exit
Enter your choice: 1
Enter a positive integer: 124356
Digits of 124356 are not in increasing order.

— DIGIT OPERATIONS MENU —
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
5. Exit
Enter your choice: 4
Enter a positive integer: 7777
The number does not contain at least two distinct digits.

— DIGIT OPERATIONS MENU —
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
5. Exit
Enter your choice: 5
Program terminated.
```

Decreasing-order test: 975310 (Pass)

```
— DIGIT OPERATIONS MENU —
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
5. Exit
Enter your choice: 2
Enter a positive integer: 975310
Digits of 975310 are in decreasing order.
```

Decreasing-order negative test: 975510 (Pass)

```
— DIGIT OPERATIONS MENU —  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
5. Exit  
Enter your choice: 2  
Enter a positive integer: 975510  
Digits of 975510 are not in decreasing order.
```

Largest and smallest digits: 508274 (Pass)

```
— DIGIT OPERATIONS MENU —  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
5. Exit  
Enter your choice: 3  
Enter a positive integer: 508274  
Largest digit = 8  
Smallest digit = 0
```

Second largest and second smallest: 583172 (Pass)

```
— DIGIT OPERATIONS MENU —  
1. Check increasing order  
2. Check decreasing order  
3. Find largest and smallest digit  
4. Find second largest and second smallest digit  
5. Exit  
Enter your choice: 4  
Enter a positive integer: 583172  
Second largest digit = 7  
Second smallest digit = 2
```

Repeated-digit verification: 7777 (Pass)

```
(kali㉿kali)-[~/hdoc16]
$ ./day16

— DIGIT OPERATIONS MENU —
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
5. Exit
Enter your choice: 1
Enter a positive integer: 124356
Digits of 124356 are not in increasing order.

— DIGIT OPERATIONS MENU —
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
5. Exit
Enter your choice: 4
Enter a positive integer: 7777
The number does not contain at least two distinct digits.

— DIGIT OPERATIONS MENU —
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
5. Exit
Enter your choice: 5
Program terminated.
```

Exit option: program terminated

```
— DIGIT OPERATIONS MENU —
1. Check increasing order
2. Check decreasing order
3. Find largest and smallest digit
4. Find second largest and second smallest digit
5. Exit
Enter your choice: 5
Program terminated.
```

Final Verification Note

All required menu options have been documented with execution evidence. Test Case 7 verifies menu choice 4 with input 7777 and confirms that the program correctly reports when at least two distinct digits are not present.